

FACULTY OF COMPUTERS AND ARTIFICIAL INTELLIGENCE, CAIRO UNIVERSITY

CS213: Programming II
Year 2026-2027

First Semester

Assignment 2 - Version 4.0

Course Instructors:
Dr. Mohammad El-Ramly

Revision History

Version 1.0	By Dr Mohammed El-Ramly	26 Oct.2022	Main Doc
Version 2.0	By Dr Mohammed El-Ramly	26 Oct.2023	Updated for 22/23
Version 3.0	By Dr Mohammed El-Ramly	2 Oct.2024	Updated for 24/25
Version 4.0	By Dr Mohammed El-Ramly	16 Oct.2027	Updated for AI 26/27

Objectives

This assignment aims to help you learn OOP concepts in C++ and how to use OOP modeling and design with C++ to build systems with intermediate complexity.

Instructions

1. **Your Learning Is YOUR Responsibility. Search for all resources that help you.**
2. Deadline for part 1 is 26 of Oct 2026 @ 11:00 pm (Tasks 1)
3. Deadline for part 2 is 6 of November 2026 @ 11:00 pm (All tasks and bonus)
4. Students form teams of **four students from the same lab/section. No mixed-section teams.**
5. **Please submit only original work that you did yourself**. If you copy work from your friend or book or AI (unless allowed), you will fail the course.
6. **Use of AI is required for Task 1 and FORBIDDEN on Task 2 and allowed on Task 3.**
7. Use ur brain, so it does not die. https://www.linkedin.com/posts/mikerosenbergwriting_lets-be-very-clear-about-whats-happening-share-7516162717603295234-NVoH/

Task 0 (0 marks)

1. Review OOP C++ concepts & syntax & code examples in the class & understand them well.
2. Read this quick tutorial https://www.codesdope.com/cpp-introduction/ or others.
3. Create a **private GitHub** repo for the project. **Use it for development.** Everyone should use it.
4. This assignment is about designing and building an OOP simulator for a fictitious machine and its machine language. It is called Vole machine. **You first need to understand Vole machine, as it is the problem domain.** It is described in pages 73 to 93 in this book Computer Science An Overview. 11th Edition by J. Glenn Brookshear.pdf (https://dn720002.ca.archive.org/0/items/gate-2020-computer-science-and-technology-by-disha-experts/Computer%20Science%20An%20Overview.%2011th%20Edition%20by%20J.%20Glenn%20Brookshear.pdf)
5. Also see: https://www.youtube.com/watch?v=fNocKCvy1EU
6. Try a Vole online simulator Machine Language Simulator / https://machinelanguage.vercel.app
7. A similar Vole simulator is https://www.youtube.com/watch?v=VC218POg0O4&t=87s
8. We are only considering implementing the instructions shown in the next table. **RFF is added.**

Machine Language (2026/27 version)

Op-code	Operand	Description
1	RXY	LOAD the register R with the bit pattern found in the memory cell whose address is XY. *Example:* 14A3 would cause the contents of the memory cell located at address A3 to be placed in register 4.
2	RXY	LOAD the register R with the bit pattern XY. *Example:* 20A3 would cause the value A3 to be placed in register 0.
3	RXY	STORE the bit pattern found in register R in the memory cell whose address is XY. *Example:* 35B1 would cause the contents of register 5 to be placed in the memory cell whose address is B1.
3	**RFF**	**STORE to location FF, which is a memory mapping for the screen. Writing to FF is writing to screen. (Added)**
4	0RS	MOVE the bit pattern found in register R to register S. *Example:* 40A4 would cause the contents of register A to be copied into register 4.
5	RST	ADD the bit patterns in registers S and T as though they were two's complement representations and leave the result in register R. *Example:* 5726 would cause the binary values in registers 2 and 6 to be added and the sum placed in register 7.
6	RST	ADD the bit patterns in registers S and T as though they represented values in floating-point notation and leave the floating-point result in register R. *Example:* 634E would cause the values in registers 4 and E to be added as floating-point values and the result to be placed in register 3.
7	RST	OR the bit patterns in registers S and Tand place the result in register R. *Example:* 7CB4 would cause the result of ORing the contents of registers B and 4 to be placed in register C.
8	RST	AND the bit patterns in registers S and Tand place the result in register R. *Example:* 8045 would cause the result of ANDing the contents of registers 4 and 5 to be placed in register 0.
9	RST	EXCLUSIVE OR the bit patterns in registers S and Tand place the result in register R. *Example:* 95F3 causes the result of EXCLUSIVE ORing the contents of registers F and 3 to be put in register 5.
A	R0X	ROTATE the bit pattern in register R one bit to the right X times. Each time place the bit that started at the low-order end at the high-order end. *Example:* A403 would cause the contents of register 4 to be rotated 3 bits to the right in a circular fashion.
B	RXY	JUMP to the instruction located in the memory cell at address XY if the bit pattern in register R is equal to the bit pattern in register number 0. Otherwise, continue with the normal sequence of execution. (The jump is implemented by copying XY into the program counter during the execute phase.) *Example:* B43C would first compare the contents of register 4 with the contents of register 0. If they were equal, pattern 3C would be placed in the program counter so that the next instruction executed would be the one located at that memory address. Otherwise, nothing would be done & program execution continue in its normal sequence.
C	000	HALT execution. *Example:* C000 would cause program execution to stop.

Task 1 (3.5 marks - Group) - OOP design

Model and design a simulator for the Vole machine and its language that is capable of simulating its operation and running the program. This design will be in the form of a UML class diagram that shows different classes, their attributes and methods and their relations. **No implementation is needed in this task.** The simulator will:

1. Offer a menu of choices as needed
2. Allow the user to load a new program from a file into a specific memory location that should be an even number (00, 02, 04, ….., F0, F2, ….) < FF_h
3. Also allow the user to overwrite the content of a given register or memory cell with a new value.
4. Program Counter (PC) is set to the start memory location of the program.
5. Then the program displays the content of memory as a matrix and the contents of IR and 16 general purpose registers.
6. The program fetches instructions a step by step to the Instruction Register (IR) and validates it as a valid step, decodes it and execute it and update PC.
7. The user can choose to run the program at once or step by step.
8. If at once, the memory, PC, IR and 16 registers and the screen display are displayed after the program finishes running.
9. If step by step, they are displayed after each step and then the user presses enter to run the next step.
10. Then the program asks the user if he wants to exit or load another program.

A program file looks like (instructions are in hex - but written as strings):

```
; ------------------------------------------------------------
; Vole Machine Program
; Purpose: Load two numbers (5 and 7), add them,
; and display the result (12) on the screen.
; ------------------------------------------------------------

2005    ; Load immediate value 05 into register 0
2107    ; Load immediate value 07 into register 1
5201    ; Add registers 0 and 1, put result in register 2
3200    ; Store register 2 into memory[00] (screen output)
C000    ; Halt
```

Note that comments start with ; and any line starting with a comment is ignored. Also any empty like is ignored. Also, each instruction is on a separate line and is only four hex digits, and the rest of the line is ignored.

Your task is as follows. Read very carefully. Failure to follow these instructions = zero automatically without any further chance.

1. You will **study the machine** very well
2. You will write **two simple sample programs** in Vole machine language that do something useful. **Class admins will create repo or shared drive and collect all these programs.** Programs should follow the format shown above. You do not need to add detailed comments; you can write a top comment or two to explain what the program does.

3. Model the Vole machine domain by **writing a list of classes** in this domain that will operate the simulator as described, the role or responsibility of each class, suggested attributes and methods of each class and the relations between classes.
4. Give your description to any AI (without A2 document or any additional input) and ask it to produce **an initial design** in PlantUML, and draw it here https://www.plantuml.com/
5. **Use three different AIs to design** the machine, separately. Feed each AI with a description of the machine and the required task and the design you suggest
6. Engage with each AI in a chat to guide it to make its best design possible and ask questions about why it did its design this way. Produce **three AI designs.**
7. Ask it to produce its design in PlantUML code & draw UML https://www.plantuml.com/
8. Take the best design of the three and ask another AI to enhance it and engage in a dialog with it till making the **final design.**
9. **Report** your experience and all the chats you had in a report **and what you learned.**
10. **Make a video** (~10 min) discussing the three designs and which AI produced the best results and how similar or different they are.
11. Submit a report 1 in pdf format named as follows and including these items:
    CS213_A2_Part1_SectionNum_YourID_YourID_YourID_YourID.pdf

    - **Failure to name the file or include any of these parts means rejecting your report.**
    - **Submitting empty files or other format = ZERO**
    - Include a cover page with names, IDs, emails, faculty and university name, assignment name, course info, team leader phone, date, etc. **Work breakdown table (who did what)**
    - Divide your reports into these sections
        - **Introduction.** Describe the vole machine and its architecture.
        - **Initial Design**. Describe the initial design you made and its classes and their details and responsibilities. Include PlantUML and UML class diagram.
        - **AI Designs.** Include the **full conversation and prompts** you had with the three AIs and the design each one produced as PlantUML code and UML class diagram.
        - **Final Design.** Include the **full conversation and prompts** you had with AI to reach and enhance final design. Describe why it is the best design.
        - **Examples.** Include the two Vole machine program examples you made to be used to evaluating the machine.
        - **Video Link**

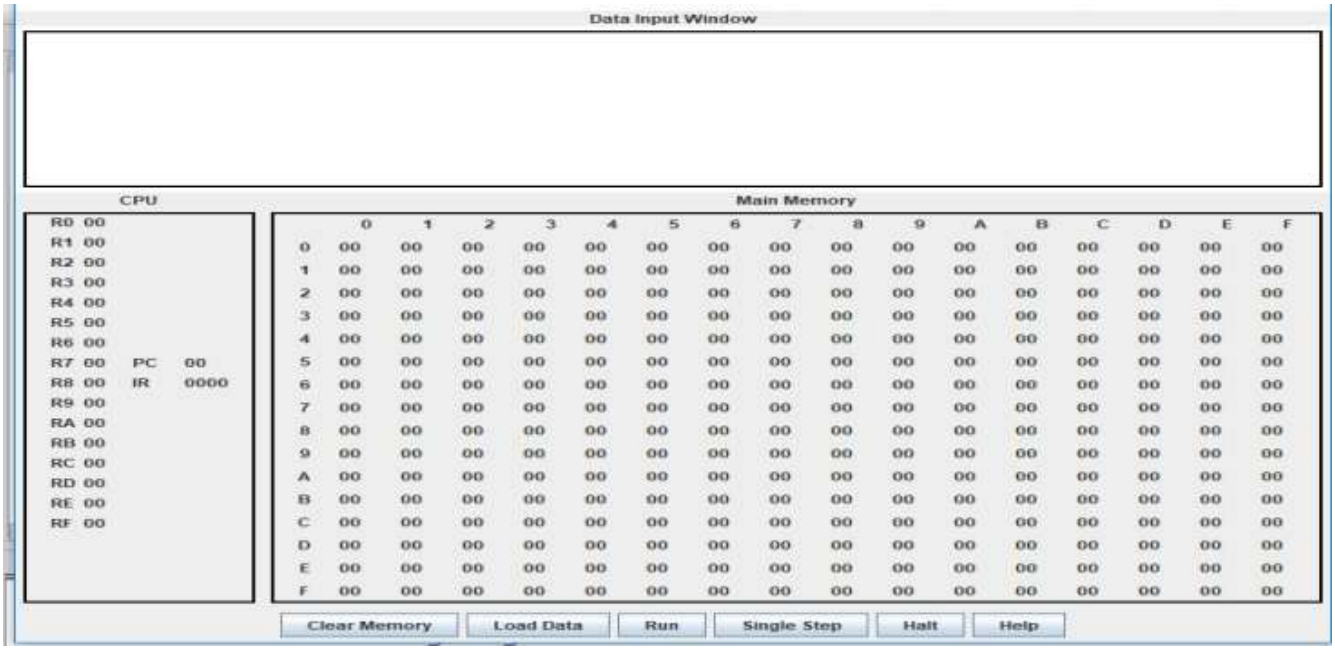
Task 3 (3.5 marks - Group) - OOP Implementation (No AI)

Implement the Vole machine simulator as described above in standard C++. Implementation should faithfully **match your final design**. It should work as described above. You need to support only the instructions given above. **Do not use AI.**

Invalid instructions are rejected and simulator should reject programs with invalid instructions. Test your program on 20 real Vole programs.

Bonus Task 3 (2 marks) - GUI for the Simulator (You can use AI)

- Team will use a GUI library for C++ (Qt, wxWidget, etc.) & develop a GUI for the simulator.
- Be creative and develop GUI that is (1) Super easy to use and (2) Intuitive to understand. Here is a sample included of previous work but it is not the best design. DO NOT copy it.



What to deliver for tasks 2 and 3?

1. **It is NOT ALLOWED to use AI in implementation of Part 2. U can use in Part 3.**
2. Students should divide the work **as they like** and then **should integrate their code** together and make sure it works properly. **They must use GitHub for collaboration.**
3. Submit your work in a file named: **CS213_A2_Part2_SectionNum_YourID_YourID_YourID_YourID.zip**
4. It should include the report u made for part 1 with an updated design. **When you code, you discover missing things and may need to update the design.**
5. Use separate compilation and divide code to header, classes and app.
6. **Working code in standard C++ not using third-party libraries if you are not doing GUI.** Name your application file **A2-YourSection-YourIDs.cpp and name your classes and header files suitable names.**
7. Add screen shots for GitHub project.
8. Add screen shots for the successful execution of 10 sample programs.
9. **Link to your video explaining the AI design process.**

If you are doing GUI

10. Include all the above and name file **CS213_A2_Part3_SectionNum_YourID_YourID_YourID_YourID.zip**
11. Include your project code (**not exe & not libraries you used**) for the GUI + Simulator.
12. Snapshots of the GUI of the simulator & README.txt on how to install & run program.
13. **A link to a 2nd public video** with FCAI logo and your name, course & prof name, explaining your GUI app and showing how to use it in clear voice.

Marking Criterion

3.5 1.5 for good final design of Vole machine and **2 for the AI design process** of moving from initial design to 3 AI designs to selecting the best to refining it to final.

3.5 For a good working implementation that loads a program to memory, starts execution (as a whole or a step by step) and displays machine status.

2 Bonus for a working GUI

-7 AI generated code or copied code.